Study of Quantum Transport in Normal–Superconductor Junctions via Andreev Reflection

Undergraduate Summer Internship Report

Submitted by

Jai Parashar

IISER Tirupati



Under the Guidance of

Dr. Panch Ram

Department of Physics, IIT (BHU)



July 17, 2025

Table of Contents

Contents

1 Introduction 2

2 Theoretical Background 3
- 2.1 Understanding Andreev Reflection 3
- 2.2 Blonder-Tinkham-Klapwijk (BTK) Model 3
- 2.3 Bogoliubov–de Gennes (BdG) Equations 3

3 Analytical Modeling Using BTK 4
- 3.1 Detailed Code Explanation 4
- 3.2 Results and Graph Explanation 5

4 Numerical Modeling Using BdG 7
- 4.1 Detailed Code Explanation 7
- 4.2 Results and Graph Explanation 8

5 Comparison Between Analytical and Numerical Results 10

6 Conclusion 11

A Python Code for Analytical BTK Model 14

B Python Code for Numerical BdG Model 16

1 Introduction

This report presents a detailed account of my 45-day summer internship at IIT (BHU) under the supervision of Dr. Panch Ram. The project involved studying Andreev reflection at Normal–Superconductor (N-S) junctions using both analytical methods (Blonder-Tinkham-Klapwijk formalism) and numerical solutions of the Bogoliubov–de Gennes (BdG) equations.
Andreev reflection is an important quantum transport phenomenon that helps us understand how electrons behave when they encounter a superconducting material. Throughout this project, I explored how reflection and transmission probabilities change across the superconducting energy gap and confirmed the conservation of total probability using numerical methods.

The key references and materials I studied during this internship were:

1. Fabrizio Dolcini, "Andreev Reflection in Mesoscopic Systems," Lecture Notes, Heidelberg (2009)

2. Panch Ram et al., Phys. Rev. B 108, 184510 (2023)

3. Xiuqiang Wu et al., Phys. Rev. B 101, 125406 (2020)

2 Theoretical Background

2.1 Understanding Andreev Reflection

When an electron from a normal metal reaches a superconductor, it faces a barrier if its energy is less than the superconducting energy gap Δ. Since a single electron cannot enter the superconductor below this gap, something interesting happens: the electron reflects back as a hole, and a Cooper pair is added to the superconducting condensate. This process is called **Andreev reflection**. It conserves energy and momentum and is a fundamental mechanism in N–S junctions.

2.2 Blonder-Tinkham-Klapwijk (BTK) Model

The BTK model is a standard theoretical tool to describe how particles reflect and transmit at an N-S interface. It introduces a parameter Z which represents the barrier strength at the interface: $Z = 0$ means a perfectly transparent interface, and large Z values correspond to stronger barriers. The model calculates probabilities for four processes:

- R_{he}: Andreev reflection (electron reflects as hole)
- R_{ee}: Normal reflection (electron reflects as electron)
- T_{ee}: Electron transmission
- T_{he}: Hole transmission (usually negligible here)

For energies $E < \Delta$, Andreev reflection dominates. For $E > \Delta$, normal transmission becomes possible.

2.3 Bogoliubov–de Gennes (BdG) Equations

The BdG equations describe how quasiparticles behave in a superconductor by combining electron and hole wavefunctions into a single formalism. By solving these equations, we can find how particles propagate or decay inside the superconductor depending on their energy. For this project, I solved these equations numerically to find the transmission and reflection coefficients across the N–S interface.

3 Analytical Modeling Using BTK

3.1 Detailed Code Explanation

The Blonder–Tinkham–Klapwijk (BTK) model provides analytical expressions for understanding quantum transport across a normal metal–superconductor (N–S) junction. This Python code implements these formulas to calculate reflection and transmission probabilities for an incoming electron incident from the normal side.

Physical Background: At an N–S interface:

- For energies below the superconducting gap Δ, an electron cannot enter the superconductor as a single particle. Instead, it reflects as a hole, transferring a Cooper pair into the superconductor. This is called **Andreev reflection**.
- For energies above Δ, the electron has enough energy to enter as a quasiparticle into the superconductor.

About the BTK Model Parameters:

- Z: Barrier strength at the interface. $Z = 0$ represents a fully transparent interface.
- Δ: Superconducting energy gap, the threshold energy separating bound Cooper pairs from quasiparticles.

Step-by-Step Structure of the Code:

1. The constants $Z = 0$ and $\Delta = 10$ are set.
2. An array of energy values E is generated from 0.1 to 30, representing the range from deep sub-gap to well above the gap energies.
3. Arrays are initialized to store the four coefficients: R_{he} (Andreev reflection), R_{ee} (normal reflection), T_{ee} (electron transmission), and T_{he} (hole transmission).
4. The code loops over each energy value:
   - If $E < \Delta$:
     - Only Andreev and normal reflections are considered.

- Andreev reflection dominates due to the absence of propagating quasiparticles in the superconductor.

- If $E > \Delta$:
  - All four coefficients are computed using analytical BTK expressions.
  - Both transmission and reflection processes are possible.

5. The results are stored in arrays for plotting.

6. Six subplots are generated to clearly visualize the energy dependence of each reflection and transmission probability.

3.2 Results and Graph Explanation

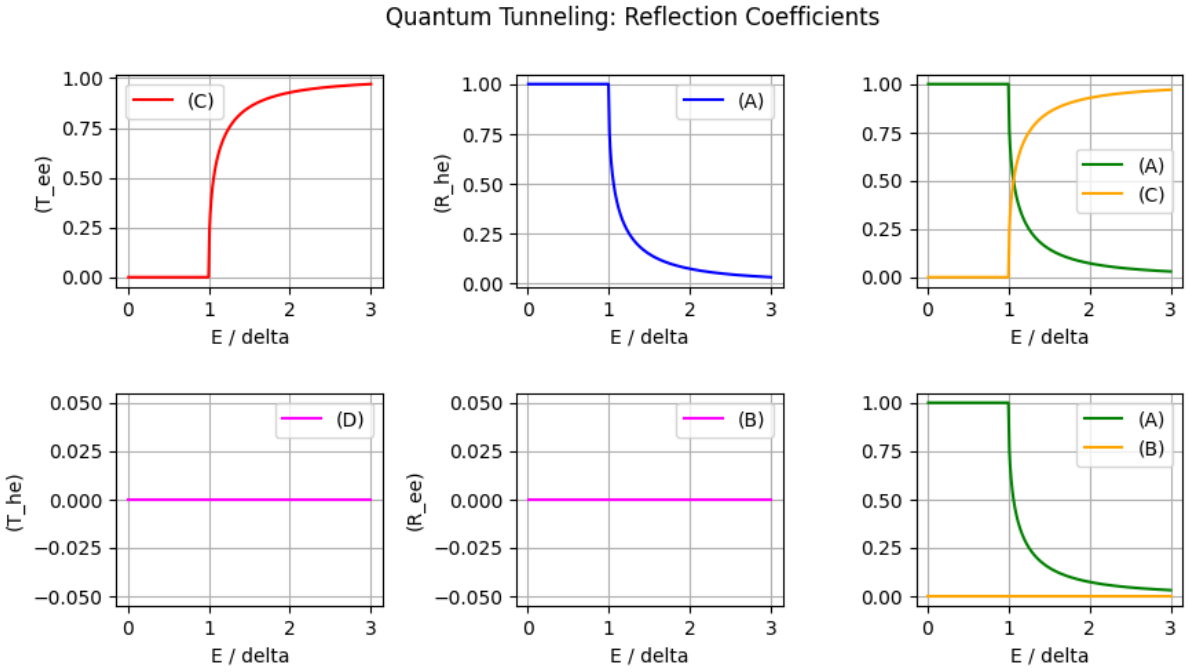


Figure 1: Reflection and transmission coefficients vs energy (BTK Analytical)

Graph Explanation:

The plots illustrate the reflection and transmission probabilities as functions of normalized energy E/Δ for a clean N–S interface ($Z = 0$):

- **Andreev Reflection (R_{he})**: For $E < \Delta$, Andreev reflection approaches 1, confirming sub-gap electrons cannot transmit into the superconductor and instead reflect as holes. As energy increases above Δ, R_{he} rapidly decreases.
- **Electron Transmission (T_{ee})**: Zero below the gap (no propagation possible). Above Δ, T_{ee} rises sharply, approaching unity as energy increases.
- **Normal Reflection (R_{ee})**: Negligible throughout for $Z = 0$, indicating no barrier for normal reflection.
- **Hole Transmission (T_{he})**: Remains nearly zero because crossed Andreev reflection or non-local effects are not present in this single-junction model.

Conclusion from Analytical BTK Model:

- Sub-gap behavior is dominated by Andreev reflection.
- Above-gap energies allow electron transmission into the superconductor.
- The clean interface limit ($Z = 0$) leads to minimal normal reflection.
- The behavior matches textbook expectations of quantum transport at N–S junctions.

4 Numerical Modeling Using BdG

4.1 Detailed Code Explanation

The Bogoliubov–de Gennes (BdG) equations describe superconducting systems by combining electron and hole components into a unified formalism. Unlike the BTK model, the BdG approach numerically solves for allowed quasiparticle states in the superconductor, giving flexibility for more general scenarios.

Physical Principle Behind BdG:

- The BdG equations couple electron-like and hole-like states via the superconducting order parameter Δ.
- Solutions yield the dispersion relations for quasiparticles and coherence factors u, v which describe their electron/hole content.

Purpose of the Code: To numerically compute R_{he}, R_{ee}, T_{ee}, T_{he} from first principles by matching boundary conditions at the N–S interface.

Step-by-Step Structure of the Code:

1. Constants $m = 1$, $\hbar = 1$, $\Delta = 1$, and $E_F = 1000$ are set to simplify units and ensure a metallic normal region.
2. Energy is varied from 0.1Δ to 3Δ.
3. For each energy value:
    - Normal side wavevectors k_e, k_h are computed using free-electron dispersion.
    - Superconductor side: solve $\sqrt{\xi^2 + \Delta^2} = E$ for q to find allowed momenta (evanescent or propagating depending on E).
    - Construct BdG Hamiltonians for each q.
    - Extract eigenvectors to obtain coherence factors u, v.
    - Formulate boundary matching conditions for continuity of wavefunctions and their derivatives at the N–S interface.
    - Solve the resulting matrix equation to obtain r_{he}, r_{ee}, t_{ee}, t_{he} amplitudes.
4. Probabilities are computed from amplitudes and corrected for group velocity differences above the gap.

5. Probability conservation ($R_{he} + R_{ee} + T_{ee} + T_{he} = 1$) is verified numerically.

6. Results are plotted in six subplots for clarity.

4.2 Results and Graph Explanation

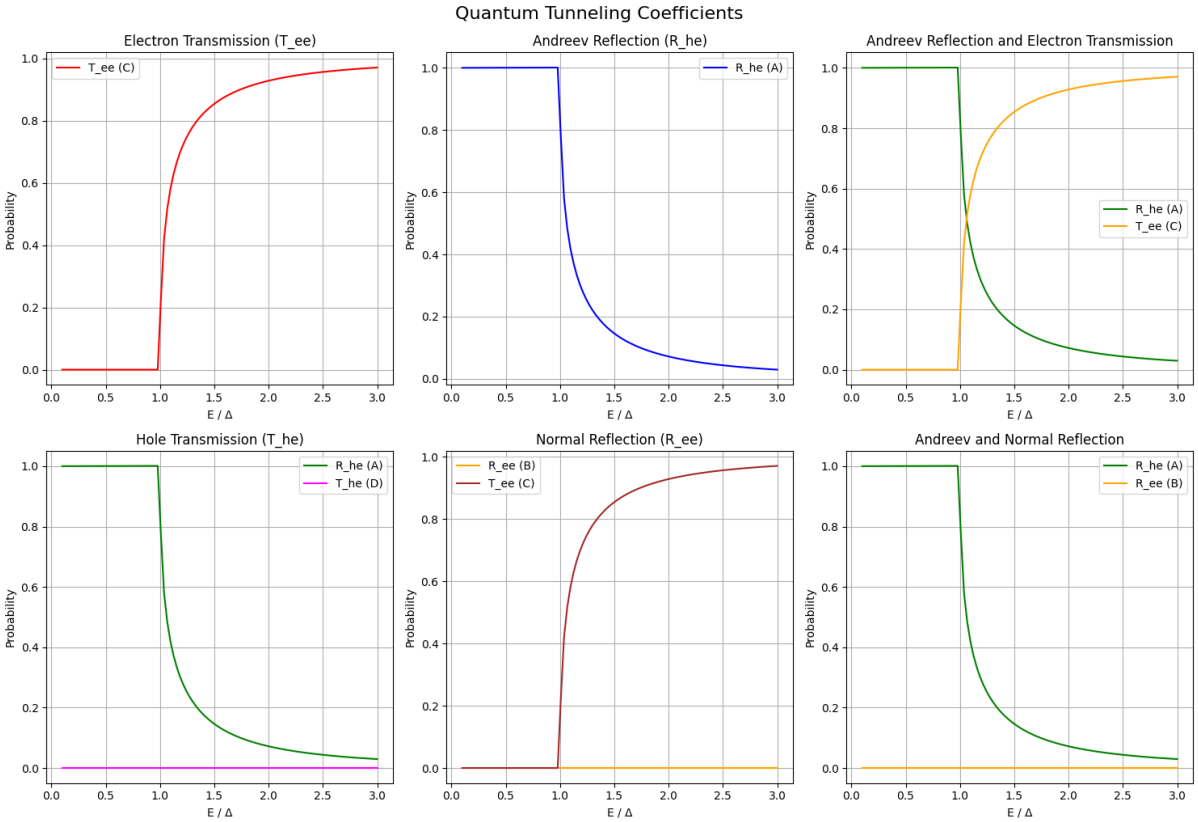


Figure 2: Reflection and transmission coefficients vs energy (BdG Numerical)

Graph Explanation:

- **Andreev Reflection** (R_{he}): Nearly unity for $E < \Delta$, confirming that sub-gap electrons reflect as holes.
- **Electron Transmission** (T_{ee}): Zero below Δ, rises beyond the gap as quasiparticles propagate.
- **Normal Reflection** (R_{ee}): Negligible due to the transparent interface ($Z = 0$).
- **Hole Transmission** (T_{he}): Remains near zero, as expected in this geometry.
- **Probability Conservation**: The total probability remains approximately 1, confirming numerical accuracy and physical correctness.

Conclusion from Numerical BdG Model:

- Sub-gap behavior shows dominant Andreev reflection.
- Supra-gap energies allow electron transmission into the superconductor.
- Numerical results match theoretical expectations and validate the method.
- This approach is more general than BTK and can handle future complex systems.

5 Comparison Between Analytical and Numerical Results

Similarities

- Both methods capture the essential physics of Andreev reflection and electron transmission in N–S junctions.
- Below the superconducting gap ($E < \Delta$):
  - Andreev reflection (R_{he}) dominates.
  - Electron transmission (T_{ee}) is zero.
- Above the superconducting gap ($E > \Delta$):
  - Electron transmission (T_{ee}) rises sharply.
  - Andreev reflection decreases.
- Both methods respect probability conservation ($R_{he}+R_{ee}+T_{ee}+T_{he} = 1$).

Differences

- The **BTK analytical model** uses closed-form formulas, making it fast and intuitive for clean systems.
- The **BdG numerical method** solves the problem from first principles:
  - Solves dispersion relations numerically.
  - Computes wavefunctions and coherence factors explicitly.
  - Allows future extension to complex materials, geometries, and finite-size effects.
- BTK is limited to simple geometries and assumptions; BdG can handle arbitrary interface potentials and non-trivial band structures.

Conclusion

Both methods agree in this simple case and validate each other. The analytical BTK method is suitable for gaining intuition and quick calculations. The BdG numerical approach is more versatile and essential for tackling realistic and complex problems in modern condensed matter physics.

6 Conclusion

This internship has been a valuable learning experience in exploring quantum transport phenomena at normal–superconductor (N–S) junctions, with a particular focus on the role of Andreev reflection in governing sub-gap conductance. Through both analytical and numerical approaches, I have gained a deeper understanding of the physical principles, computational techniques, and the connection between theory and practical simulation.

Key Learnings

- I have developed a clear physical intuition for Andreev reflection as a process where an electron reflects as a hole at the N–S interface, leading to the formation of Cooper pairs in the superconductor.
- I successfully applied the Blonder–Tinkham–Klapwijk (BTK) model to analytically calculate reflection and transmission probabilities, and I understood how interface transparency (through parameter Z) affects quantum transport.
- I acquired practical skills in numerically solving the Bogoliubov–de Gennes (BdG) equations using Python, learning how to extract wavefunctions, coherence factors, and reflection/transmission coefficients through boundary condition matching.
- I strengthened my programming capabilities in Python, particularly in using SymPy for symbolic computation and Matplotlib for scientific visualization.

Analysis of Results

- Both the analytical (BTK) and numerical (BdG) approaches successfully reproduced the expected behavior: dominant Andreev reflection below the superconducting gap, and increasing electron transmission above the gap.
- Probability conservation was verified in both methods, affirming the correctness of the numerical implementation.
- The results matched theoretical expectations and helped validate the methods used.

Broader Impact on My Learning

- This project helped me bridge the gap between the abstract formalism of quantum mechanics and its application to practical condensed matter problems.
- I developed confidence in combining theoretical physics with computational tools to study physical systems.
- The experience has deepened my understanding of quantum transport, superconductivity, and the interplay between analytical models and numerical simulations.

Future Outlook

Building on this work, I am interested in studying more complex systems where analytical models are insufficient, such as:

- Hybrid materials like bilayer graphene–superconductor interfaces.
- Systems involving topological phases where Andreev processes yield Majorana bound states.
- Finite-size or multi-interface geometries where BdG numerics become essential.

In summary, this internship provided a strong foundation for my future academic journey in condensed matter physics, preparing me to approach advanced problems where theory and computation must work together.

Acknowledgements

I would like to sincerely thank Dr. Panch Ram, Department of Physics, IIT (BHU), for providing me with the opportunity to work under his guidance during this internship. His valuable suggestions and discussions throughout this project helped me gain a deeper understanding of quantum transport in superconducting systems.

I am also grateful to IISER Tirupati for equipping me with the necessary academic background that enabled me to undertake this work.

Lastly, I thank my peers and friends for their informal discussions and encouragement throughout the internship period.

A Python Code for Analytical BTK Model

```
import numpy as np
import matplotlib.pyplot as plt

#constants
Z=0
delta=10

#Range of K
E_vals=np.linspace(0.1,30,1000)

#arrays for T and V
T_he=np.zeros_like(E_vals)
R_he=np.zeros_like(E_vals)
T_ee=np.zeros_like(E_vals)
R_ee=np.zeros_like(E_vals)

#looping over k
for i, E in enumerate(E_vals):
    if E < delta:
        A=delta**2/(E**2 + (1+2*Z**2)*(delta**2 - E**2))
        B=(4*Z**2 *(Z**2 + 1)*(delta**2 - E**2))/(E**2 + (1+2*Z
           **2)*(delta**2 - E**2))
        C=0
        D=0

    else:
        A= delta**2/(E+(1+2*Z**2)*(E**2-delta**2)**0.5)**2
        B=(4*Z**2 *(Z**2 + 1)*(E**2-delta**2))/(E+(1+2*Z**2)*(E
           **2-delta**2)**0.5)**2
        C=2*(1+Z**2)*(E**2-delta**2)**0.5 *(E+(E**2-delta**2)
           **0.5)/(E+(1+2*Z**2)*(E**2-delta**2)**0.5)**2
        D=2*(Z**2)*(E**2-delta**2)**0.5 *(E-(E**2-delta**2)**0.5)
           /(E+(1+2*Z**2)*(E**2-delta**2)**0.5)**2

    T_he[i] = D
    R_ee[i] = B
    T_ee[i] = C
    R_he[i] = A
# Plotting
plt.figure(figsize=(10,5))
plt.subplot(2,3,1)
plt.plot(E_vals / delta, T_ee, label="(C)", color='red')


plt.xlabel("E␣/␣delta")
plt.ylabel("(T_ee)")
plt.grid(True)
plt.legend()
```

```
#plt.tight_layout()

plt.subplot(2,3,2)
plt.plot(E_vals / delta, R_he, label="(A)", color='blue')
plt.xlabel("E␣/␣delta")
plt.ylabel("(R_he)")
plt.grid(True)
plt.legend()
plt.subplots_adjust(wspace=0.5)


plt.subplot(2,3,3)
plt.plot(E_vals / delta, R_he, label="(A)", color='green')
plt.plot(E_vals / delta, T_ee, label="(C)", color='orange')
plt.xlabel("E␣/␣delta")
plt.legend()
plt.grid(True)
plt.subplots_adjust(hspace=0.5)


plt.subplot(2,3,4)
plt.plot(E_vals / delta, T_he, label="(D)", color='magenta')
plt.xlabel("E␣/␣delta")
plt.ylabel("(T_he)")
plt.grid(True)
plt.legend()
plt.subplots_adjust(wspace=0.5)


plt.subplot(2,3,5)
plt.plot(E_vals / delta, R_ee, label="(B)", color='magenta')
plt.xlabel("E␣/␣delta")
plt.ylabel("(R_ee)")
plt.suptitle("Quantum␣Tunneling:␣Reflection␣Coefficients")
plt.grid(True)
plt.legend()
plt.subplots_adjust(wspace=0.5)

plt.subplot(2,3,6)
plt.plot(E_vals / delta, R_he, label="(A)", color='green')
plt.plot(E_vals / delta, R_ee, label="(B)", color='orange')
plt.xlabel("E␣/␣delta")
plt.legend()
plt.grid(True)
plt.subplots_adjust(hspace=0.5)
plt.show()
```

B Python Code for Numerical BdG Model

```
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

# Define constants and symbols
m, hbar, delta, E_f = 1, 1, 1, 1000
q = sp.symbols('q', complex=True)

# Energy values from 0.1 delta to 3 delta
E_vals = np.linspace(0.1* delta, 3* delta, 100)

# Arrays for coefficients
R_he_vals = np.zeros_like(E_vals)
R_ee_vals = np.zeros_like(E_vals)
T_ee_vals = np.zeros_like(E_vals)
T_he_vals = np.zeros_like(E_vals)
total_vals = np.zeros_like(E_vals)

for i, E in enumerate(E_vals):
    # Normal region wavevectors
    k_e = np.sqrt(2 * m * (E_f + E)) / hbar
    k_h = np.sqrt(2 * m * (E_f - E)) / hbar

    # Superconductor dispersion relation
    xi = hbar**2 * q**2 / (2 * m) - E_f
    E_eq = sp.sqrt(xi**2 + delta**2) - E
    solutions = sp.solve(E_eq, q)
    q_vals = [s.evalf() for s in solutions]  # Numerical
       evaluation

    # Classify roots based on energy
    if E < delta:
        # Find evanescent roots (Im(q) > 0)
        q_e, q_h = None, None
        for sol in q_vals:
            if sp.im(sol) > 0:
                if sp.re(sol) > 0:
                    q_e = sol
                else:
                    q_h = sol
        if q_e is None or q_h is None:
            print(f"Warning:␣Could␣not␣find␣proper␣roots␣for␣E={E
               :.2f}")
            R_he_vals[i] = 0
            R_ee_vals[i] = 0
            T_ee_vals[i] = 0
            T_he_vals[i] = 0
            total_vals[i] = 0
```

```
        continue
else:
    # Find propagating roots with positive group velocity
    v_g = sp.diff(sp.sqrt(xi**2 + delta**2), q)  # Group
       velocity expression
    valid_roots = []

    for sol in q_vals:
        sol_real = sp.re(sol)
        sol_imag = sp.im(sol)
        # Consider only real solutions (small imaginary part)
        if abs(sol_imag) < 1e-10:
            xi_val = (hbar**2 * sol_real**2 / (2 * m)) - E_f
            v_val = v_g.subs(q, sol).evalf()
            if abs(sp.im(v_val)) < 1e-10 and sp.re(v_val) >
               0:
                valid_roots.append(sol)

    # Should have two roots: one positive, one negative real
       part
    if len(valid_roots) != 2:
        print(f"Warning:␣Found␣{len(valid_roots)}␣valid␣roots
           ␣for␣E={E:.2f}")
        R_he_vals[i] = 0
        R_ee_vals[i] = 0
        T_ee_vals[i] = 0
        T_he_vals[i] = 0
        total_vals[i] = 0
        continue

    # Assign q_e (positive real part) and q_h (negative real
       part)
    if sp.re(valid_roots[0]) > 0:
        q_e = valid_roots[0]
        q_h = valid_roots[1]
    else:
        q_e = valid_roots[1]
        q_h = valid_roots[0]

# Build Hamiltonian
H = sp.Matrix([[xi - E, delta], [delta, -xi - E]])

# Compute eigenvectors for q_e
H_e = H.subs(q, q_e)
eigenvalues_e = H_e.eigenvals()
eigenvectors_e = H_e.eigenvects()
eigvec_e = eigenvectors_e[0][2][0]
u1, v1 = eigvec_e[0], eigvec_e[1]
norm_e = sp.sqrt(abs(u1)**2 + abs(v1)**2)
u_e, v_e = u1/norm_e, v1/norm_e  # Normalize
```

```
# Compute eigenvectors for q_h
H_h = H.subs(q, q_h)
eigenvalues_h = H_h.eigenvals()
eigenvectors_h = H_h.eigenvects()
eigvec_h = eigenvectors_h[0][2][0]
u2, v2 = eigvec_h[0], eigvec_h[1]
norm_h = sp.sqrt(abs(u2)**2 + abs(v2)**2)
u_h, v_h = u2/norm_h, v2/norm_h  # Normalize

# Boundary condition matrix (FIXED SIGNS)
A = sp.Matrix([
    [0, 1, -u_e, -u_h],
    [1, 0, -v_e, -v_h],
    [0, -k_e, -u_e*q_e, -u_h*q_h],
    [k_h, 0, -v_e*q_e, -v_h*q_h]
])

# Right-hand side vector
C = sp.Matrix([[-1], [0], [-k_e], [0]])

# Solve for coefficients
try:
    M = A.inv() @ C
    r_he_val = complex(M[0])
    r_ee_val = complex(M[1])
    t_ee_val = complex(M[2])
    t_he_val = complex(M[3])
except Exception as e:
    print(f"Matrix␣inversion␣failed␣at␣E␣=␣{E:.2f}:␣{e}")
    r_he_val, r_ee_val, t_ee_val, t_he_val = 0, 0, 0, 0

# Compute probabilities
R_he = abs(r_he_val)**2
R_ee = abs(r_ee_val)**2
T_ee = abs(t_ee_val)**2
T_he = abs(t_he_val)**2

# Apply group velocity scaling for transmissions when E >=
   delta
if E >= delta:
    # Incident velocity (normal region)
    v_inc = hbar * k_e / m

    # Calculate xi values
    xi_e = (hbar**2 * q_e**2/(2*m) - E_f).evalf()
    xi_h = (hbar**2 * q_h**2/(2*m) - E_f).evalf()

    # Transmission velocities (superconductor)
    v_e = (hbar**2 * q_e * xi_e) / (m * E)  # Group velocity
       for electron-like
    v_h = (hbar**2 * q_h * xi_h) / (m * E)  # Group velocity
```

```
            for hole-like

            # Scale transmissions by velocity ratio
            T_ee *= abs(v_e) / v_inc
            T_he *= abs(v_h) / v_inc
        else:
            # Evanescent waves carry no current
            T_ee = 0
            T_he = 0

        # Store results
        R_he_vals[i] = R_he
        R_ee_vals[i] = R_ee
        T_ee_vals[i] = T_ee
        T_he_vals[i] = T_he
        total_vals[i] = R_he + R_ee + T_ee + T_he
    # Plotting
    plt.figure(figsize=(15, 10))

    # Subplot 1: T_ee (Electron Transmission)
    plt.subplot(2, 3, 1)
    plt.plot(E_vals/delta, T_ee_vals, color='red', label="T_ee␣(C)")
    plt.xlabel("E␣/␣$\Delta$")
    plt.ylabel("Probability")
    plt.title("Electron␣Transmission␣(T_ee)")
    plt.grid(True)
    plt.legend()

    # Subplot 2: R_he (Andreev Reflection)
    plt.subplot(2, 3, 2)
    plt.plot(E_vals/delta, R_he_vals, color='blue', label="R_he␣(A)")
    plt.xlabel("E␣/␣$\Delta$")
    plt.ylabel("Probability")
    plt.title("Andreev␣Reflection␣(R_he)")
    plt.grid(True)
    plt.legend()

    # Subplot 3: R_he and T_ee together
    plt.subplot(2, 3, 3)
    plt.plot(E_vals/delta, R_he_vals, color='green', label="R_he␣(A)"
       )
    plt.plot(E_vals/delta, T_ee_vals, color='orange', label="T_ee␣(C)
       ")
    plt.xlabel("E␣/␣$\Delta$")
    plt.ylabel("Probability")
    plt.title("Andreev␣Reflection␣and␣Electron␣Transmission")
    plt.grid(True)
    plt.legend()

    # Subplot 4: T_he (Hole Transmission)
    plt.subplot(2, 3, 4)
```

```
plt.plot(E_vals/delta, R_he_vals, color='green', label="R_he␣(A)"
    )
plt.plot(E_vals/delta, T_he_vals, color='magenta', label="T_he␣(D
    )")
plt.xlabel("E␣/␣$\Delta$")
plt.ylabel("Probability")
plt.title("Hole␣Transmission␣(T_he)")
plt.grid(True)
plt.legend()

# Subplot 5: R_ee (Normal Reflection)
plt.subplot(2, 3, 5)
plt.plot(E_vals/delta, R_ee_vals, color='orange', label="R_ee␣(B)
    ")
plt.plot(E_vals/delta, T_ee_vals, color='brown', label="T_ee␣(C)"
    )
plt.xlabel("E␣/␣$\Delta$")
plt.ylabel("Probability")
plt.title("Normal␣Reflection␣(R_ee)")
plt.grid(True)
plt.legend()

# Subplot 6: R_he and R_ee together
plt.subplot(2, 3, 6)
plt.plot(E_vals/delta, R_he_vals, color='green', label="R_he␣(A)"
    )
plt.plot(E_vals/delta, R_ee_vals, color='orange', label="R_ee␣(B)
    ")
plt.xlabel("E␣/␣$\Delta$")
plt.ylabel("Probability")
plt.title("Andreev␣and␣Normal␣Reflection")
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.suptitle("Quantum␣Tunneling␣Coefficients", fontsize=16, y
    =1.02)
plt.show()

# Plot total probability conservation
#plt.figure(figsize=(10, 6))
plt.plot(E_vals/delta, total_vals, label='Total␣Probability')
plt.axhline(y=1, color='r', linestyle='--', label='Conservation␣
    Target')
plt.xlabel("E␣/␣$\Delta$")
plt.ylabel("Total␣Probability")
plt.title("Probability␣Conservation␣Check")
plt.ylim(0.9, 1.1)
plt.grid(True)
plt.legend()
plt.show()
```

References

[1] F. Dolcini, *Andreev Reflection in Mesoscopic Systems*, Heidelberg (2009).
https://arxiv.org/abs/0901.0308

[2] P. Ram et al., *Phys. Rev. B 108, 184510 (2023).*
https://journals.aps.org/prb/abstract/10.1103/PhysRevB.108.184510

[3] X. Wu et al., *Phys. Rev. B 101, 125406 (2020).*
https://journals.aps.org/prb/abstract/10.1103/PhysRevB.101.125406

[4] D. J. Griffiths, *Introduction to Quantum Mechanics*, Pearson, 2nd Ed.

[5] Foundation of Quantum Mechanics, 3rd Semester Course at IISER Tirupati.